Relatório - Implementação 5: Sincronização

Aluno	Luiz Henrique Carvalho Vieira
Professor	Marcelo Lima Arcoverde
Turma	3º período B
Disciplina	Infraestrutura de software
Login	lhcv
Repositório	github.com/luizvieiraa/Sync

1. Objetivo

Desenvolver em C a sincronização de carros de metrô e passageiros, controlados por threads. Entregar as quatro funções de atendimento da estação, garantindo limite de assentos, embarques simultâneos e partida somente após as confirmações necessárias, além de Makefile, testes, README e evidências de execução.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Implementação em C para Linux/Unix/macOS	OK	Compilação e execução verificadas no Ubuntu/WSL2
struct station e quatro funções públicas	OK	sync_car.h e sync_car.c
Primitivas pthreads exigidas	OK	init, lock/unlock, wait, signal e broadcast no código
Um único lock por estação	OK	Um pthread_mutex_t em struct station
Sem semáforos ou outras primitivas	OK	Inspeção de sync_car.c e sync_car.h
Um carro por vez e mesmo destino	OK	Premissas da interface; runner sequencia os carros
Embarques simultâneos	OK	Runner original e reserva com mutex liberado antes do embarque
Sem busy-wait na implementação	OK	Esperas com pthread_cond_wait e while
Respeitar vagas e aguardar confirmações	OK	Runner original com 100 passageiros
Makefile gera sync_car e permite limpeza	OK	make clean; make; ./sync_car
README e relatório no GitHub	OK	README.md e lhcv.pdf na raiz do repositório
Pasta lhcv e pacote lhcv.tar	OK	Pacote com fontes, Makefile, README e evidencias.log

3. Como Reproduzir

Ambiente utilizado: Ubuntu 26.04 LTS no WSL2, GCC 15.2.0 e GNU Make 4.4.1. É necessário um compilador C com pthreads e Make. A validação foi realizada no Ubuntu; Linux/Unix/macOS são os ambientes previstos pelo enunciado.

A partir da raiz do repositório:

```
cd lhcv
make clean
make
./sync_car
```

Para reproduzir pelo pacote, execute tar -xf lhcv.tar, entre na pasta lhcv e use os mesmos comandos a partir de make clean. O runner não recebe argumentos e deve terminar com Looks good!. As vagas por carro variam porque são escolhidas aleatoriamente.

O runner fornecido pelo professor não recebe argumentos nem arquivo de entrada. A implementação não acrescenta mensagens, opções ou validações externas à interface solicitada.

4. Arquitetura (resumida)

Arquivos e responsabilidades:

Arquivo	Responsabilidade
sync_car.c	Implementa as quatro funções de sincronização da estação.
sync_car.h	Expõe struct station e os protótipos usados pelo runner.
sync_car-runner.c	Runner original do professor, preservado; contém main e 100 passageiros.
Makefile	Compila o executável sync_car e limpa executáveis e objetos.
evidencias.log e evidencias/	Guardam sessões automáticas e capturas de terminal.

Decisões técnicas:

Decisão	Motivo	Consequência
Um mutex por estação	Proteger todo o estado	Contadores alterados de forma exclusiva
Duas condições	Separar vagas de confirmações	Passageiros e carro esperam eventos diferentes
Reserva antes do retorno	Evitar exceder os assentos	Embarque físico ocorre com mutex liberado
While nas esperas	Reavaliar o estado ao acordar	Despertares não autorizam ações sem conferir vagas
Estrutura no cabeçalho	Runner instancia struct station	Tipo completo disponível sem alterar o runner

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca
S1	Mutex e contadores	Uma fila lógica, vagas e embarques pendentes, com duas condições	Não houve troca de abordagem; o desenvolvimento foi dividido por função

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa	Quando	Evidência
1	S1	Inicializar estrutura e pthreads	OK	-	28/10 22:15	c024e68; log
2	S1	Copiar runner e reservar vagas na espera	OK	-	28/10 22:27	9ac5cb2; log
3	S1	Confirmar embarque e sinalizar conclusão	OK	-	31/10 00:45	36d8ae8; log
4	S1	Controlar chegada e partida; executar runner	OK	-	31/10 00:46	cd96a88; log
5	S1	Criar Makefile e testes limite/inválido	OK	-	03/11 19:38	d63b24e; log
6	S1	Executar versão com && na espera	Falha	Espera exigia as duas condições ao mesmo tempo	03/11 19:44	Figura 1
7	S1	Executar com || restaurado	OK	-	03/11 19:47	Figura 2
8	S1	Recompilar do zero e executar toda a suíte	OK	-	03/11 19:51	Log: validação final
9	S1	Compilar e testar a cópia extraída do pacote	OK	-	03/11 19:55	Log: reprodução do pacote extraído
10	S1	Auditar a entrega e corrigir identificação para Implementação 5	OK	-	03/11 19:59	Log: auditoria pré-entrega
11	S1	Retirar extras e validar apenas com o runner original, inclusive no pacote extraído	OK	-	03/11 20:09-20:10	Log: entrega e reprodução apenas runner professor

As linhas 6 e 7 documentam uma comparação de versões para depuração. Registrei as capturas no terminal; os horários correspondem aos nomes dos arquivos originais. As demais linhas correspondem às sessões automáticas do evidencias.log.

A principal dificuldade de sincronização é distinguir uma vaga reservada de um embarque confirmado. Se o carro considerar apenas o número de vagas, pode partir enquanto ainda há pessoas entrando. Por isso, o estado mantém um contador independente de embarques pendentes.

A integração também exigiu expor a estrutura no cabeçalho e preservar o runner original. O aviso de parâmetro não utilizado do runner foi tratado no Makefile, apenas na compilação desse arquivo. Não ficou função solicitada sem implementação; as limitações operacionais estão descritas no README e na seção final. Os testes auxiliares registrados no histórico foram retirados da entrega, junto com a asserção extra para assentos negativos. A versão final é validada pelo runner fornecido pelo professor.

6. Evidências

6.1 Log automático

O arquivo evidencias.log acompanha o código no .tar. As sessões foram registradas com script desde 28/10/2026. A sessão inicial contém date, whoami e pwd; algumas sessões intermediárias repetem apenas date. As validações finais registram os três comandos. O log original foi preservado sem reconstruir dados ausentes. Uma validação anterior começou em 03/11/2026 às 19:51:31, no diretório /mnt/c/Users/luizv/Desktop/sync/lhcv. A sessão registra ambiente, compilação limpa, testes, hashes e limpeza final. Outra sessão, às 19:55, reproduz a compilação e os testes na cópia extraída do pacote. A auditoria pré-entrega, às 19:59, repete a verificação no pacote identificado como Implementação 5. Às 20:09 e 20:10, a entrega final, sem testes auxiliares, foi recompilada e aprovada pelo runner original, no projeto e na cópia extraída do .tar.

Verificação	Comando / cenário	Resultado observado
Ambiente	uname -sr; gcc --version; make --version	WSL2; GCC 15.2.0; Make 4.4.1
Compilação limpa	make clean; make	Executável sync_car gerado sem diagnósticos
Runner original	./sync_car	100 passageiros; Looks good!
Estação vazia / zero vagas	Chamadas iniciais do runner	Retorno imediato nos casos previstos
Capacidades variáveis	Carros aleatórios no runner	Embarque igual ao mínimo entre vagas e passageiros restantes
Runner preservado	sha256sum dos dois arquivos	Hashes idênticos ao original recebido
Limpeza	make clean	Remove executáveis e objetos, preservando fontes e evidências
Entrega extraída	make clean; make; ./sync_car na cópia do .tar	Runner aprovado na versão final

Durante o desenvolvimento, executei também testes auxiliares de reservas simultâneas e um caso inválido com -1 assento, rejeitado por uma asserção então existente. Esses resultados permanecem no log como histórico. Os testes extras e a asserção não integram a versão final, pois não foram pedidos no enunciado. O teste reproduzível da entrega é ./sync_car, sem argumentos.

Funcionamento das funções e condição de partida

station_init prepara o mutex, as duas condições e os três contadores em zero. A inicialização ocorre antes da criação das threads que usam a estação.

station_wait_for_car entra na fila e espera enquanto não há vagas. pthread_cond_wait libera o mutex ao bloquear e o readquire antes de retornar. Ao encontrar uma vaga, a função reduz assentos_disponiveis e passageiros_esperando e aumenta passageiros_embarcando. A reserva é feita sob o mesmo mutex, impedindo que duas threads ocupem a mesma vaga.

station_on_board reduz passageiros_embarcando e sinaliza embarque_concluido quando esse contador chega a zero e o carro está cheio ou não há mais passageiros esperando. station_load_car usa broadcast para anunciar vagas, aguarda embarques pendentes ou vagas com passageiros na fila, e zera as vagas antes de retornar. Dessa forma, um passageiro posterior não reserva assento de um carro que já partiu.

6.2 Prints

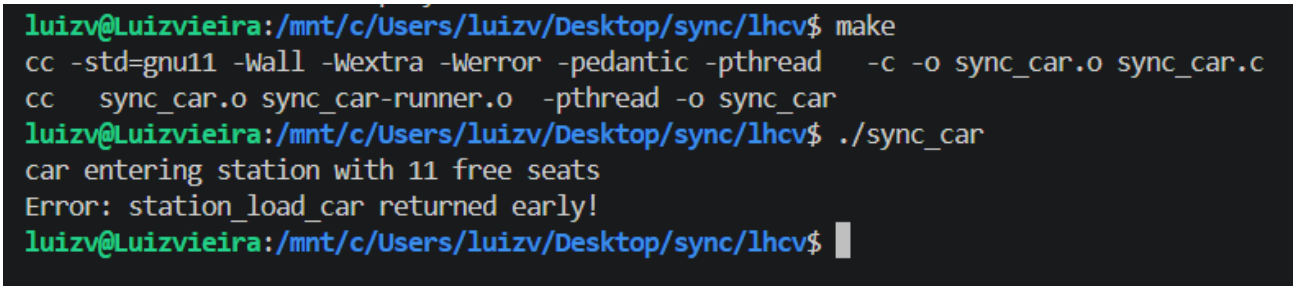


Figura 1 - O runner detecta retorno antecipado de station_load_car na versão com &&.

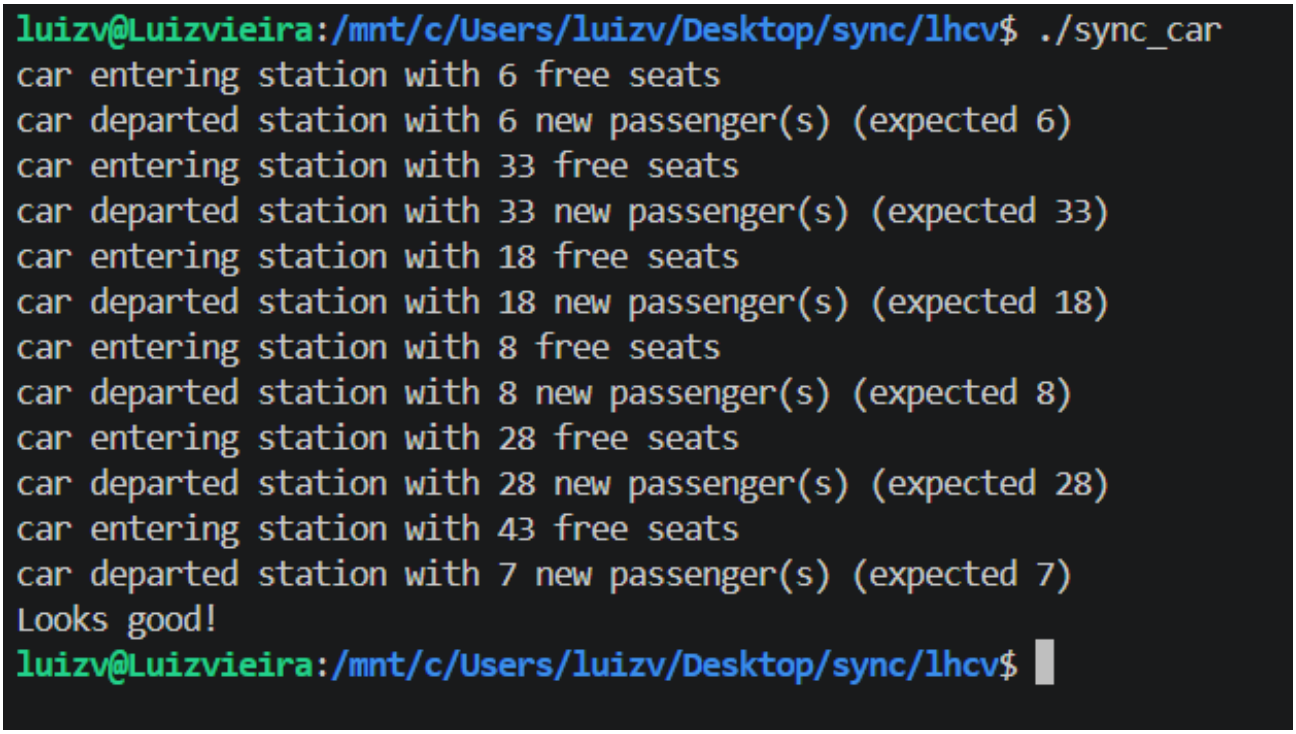


Figura 2 - Com a condição corrigida, os 100 passageiros embarcam e o runner aprova a execução.

O uso de && no while permitia que o carro partisse antes da conclusão dos embarques. A troca por || corrigiu a condição, e o runner terminou com Looks good!.

Aspecto	Análise
Sintoma	station_load_car returned early!
Causa	O operador && exigia simultaneamente embarques pendentes e vagas com fila.
Efeito	Na chegada, zero passageiros embarcando tornava a condição falsa.
Correção	Usar || para aguardar qualquer uma das condições que impedem a partida.

Histórico de commits

Commit	Descrição
c024e68	Etapa 1: estrutura da estação e inicialização
9ac5cb2	Etapa 2: runner e espera por vaga
36d8ae8	Etapa 3: confirmação de embarque
cd96a88	Etapa 4: chegada e partida do carro
d63b24e	Etapa 5: Makefile e testes de sincronização

7. Uso de IA

Utilizei IA para interpretar o enunciado, organizar as etapas, gerar e explicar as funções em C, preparar testes e Makefile e redigir o README e este relatório. Também solicitei à ferramenta a execução de comandos no WSL e a conferência dos resultados no log.

Prompts principais:

1 Ler o PDF e dividir o projeto em etapas, explicando cada função durante a implementação.

2 Implementar a estrutura e as funções de sincronização no modelo exigido pelo professor.

3 Adicionar o runner fornecido sem alterações e criar um commit por etapa.

4 Criar o Makefile e testes de casos limite e entrada inválida.

5 Preparar um exercício de depuração da condição de espera e verificar sua correção.

6 Gerar README, relatório no modelo enviado e pacote lhcv.tar.

O que validei manualmente e como:

Executei o runner no terminal, observei o retorno antecipado e identifiquei o operador incorreto, analisando por que a condição precisava de ||. Depois executei a versão corrigida e registrei as duas capturas. Solicitei à ferramenta a compilação limpa e a execução do runner no WSL; os resultados e o hash do runner estão no evidencias.log.

8. Reflexão Final

A sincronização depende de representar separadamente espera, reserva e confirmação. Um assento reservado ainda pode estar com o embarque pendente, por isso o carro não deve decidir a partida apenas pelas vagas restantes. O mutex único simplificou a consistência dos contadores, enquanto as condições permitiram bloquear sem espera ocupada. A comparação entre && e || mostrou que uma pequena alteração lógica pode invalidar a condição de partida. Reter confirmações nos testes ajudou a verificar o embarque simultâneo de forma observável. Em trabalhos seguintes, manteria o registro automático desde a primeira sessão e revisaria cada condição de espera em relação ao estado compartilhado.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim - comandos executados no WSL
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim - Figuras 1 e 2
Testei pelo menos um caso limite e um caso inválido	Sim - zero vagas no runner; inválido no histórico (seção 6)
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Eu acrescentaria testes de estresse com mais passageiros, capacidades variadas e diversas execuções para ampliar a cobertura de intercalações entre threads. Criaria um alvo separado para análise dinâmica de concorrência, avaliando as limitações do próprio runner antes de interpretar os resultados. Também testaria a compilação em macOS, pois a execução registrada foi somente no Ubuntu. Revisaria uma política de tratamento de falhas de inicialização de pthreads e de cancelamento de threads, que não foi solicitada pela interface. Por fim, incluiria um manifesto versionado dos hashes da entrega para permitir sua conferência independente após o download do repositório.

Todas as funções solicitadas foram implementadas. A solução assume um carro por vez, count não negativo e uma confirmação para cada reserva. Não oferece ordem FIFO entre passageiros; essa garantia não foi exigida. Não acrescentei uma interface de entrada ou saída além do runner fornecido. A aprovação do runner não demonstra todas as intercalações possíveis de threads.

Entrega: lhcv.pdf e lhcv.tar. O pacote contém a pasta lhcv com código-fonte, Makefile, README, evidencias.log e as capturas utilizadas neste relatório.